

Modelowanie architektury

dr hab. Michał Śmiałek, prof. uczelni
dr Kamil Rybiński



**Wydział
Elektryczny**
POLITECHNIKA WARSZAWSKA

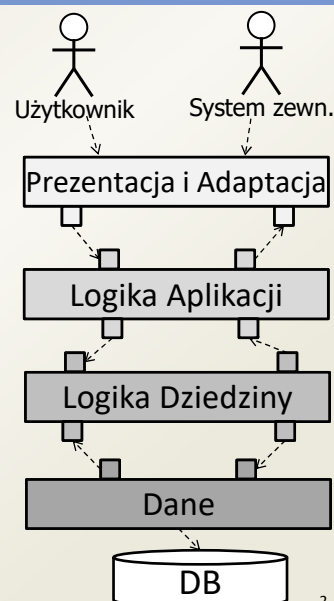
1DI1306: Modelowanie oprogramowania w języku UML



1

Przypomnienie: architektura warstwowa

- Warstwy: „odległość” od aktorów (obiektów poza systemem)
- Architektura wielowarstwowa
 - Każda warstwa komunikuje się tylko z warstwą niżej i wyżej)
 - Górne warstwy są szybko zmienne (kontrolki, formularze,...)
 - Warstwy środkowe zmieniają się wraz ze zmianą wymagań funkcjonalnych i dziedziny problemu
 - Dolne warstwy zmieniają się w miarę zmian technologii (SZBD)
- Typowe warstwy (wzorzec MVP)
 - Widok (V) - prezentacja
 - Prezenter (P) - logika aplikacji
 - Model (M) - logika dziedziny



2

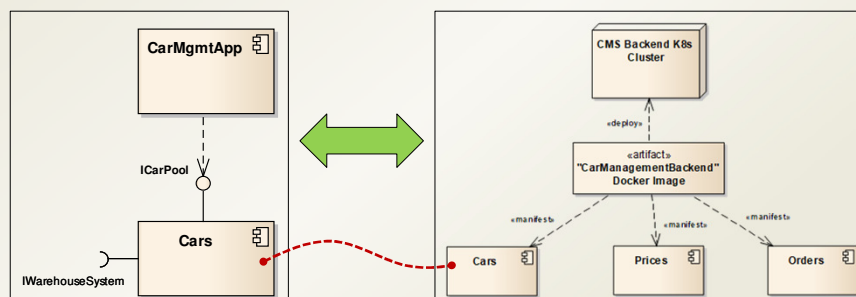
Co to jest architektura oprogramowania?

- Architektura oprogramowania
 - Decyzje podjęte na bazie wymagań klienta i wiedzy na temat technologii wytwarzania oprogramowania
 - Pokazują strukturę i dynamikę (działanie) systemu, który ma zostać (lub został) zbudowany
 - Wyrażone w języku graficznym (wizualnym) zrozumiałym dla innych projektantów i wykonawców systemu
- Architektura logiczna
 - podział na jednostki funkcjonalne realizujące logikę systemu (komponenty); współpraca w celu realizacji wymagań
- Architektura fizyczna
 - podział na jednostki fizyczne (maszyny fizyczne lub wirtualne); jednostki logiczne wykonywane są na jednostkach fizycznych

3

Architektura wyrażona w języku UML

- Diagram komponentów pokazuje architekturę logiczną
- Diagram montażu pokazuje architekturę fizyczną
- Komponenty są „manifestowane” przez artefakty



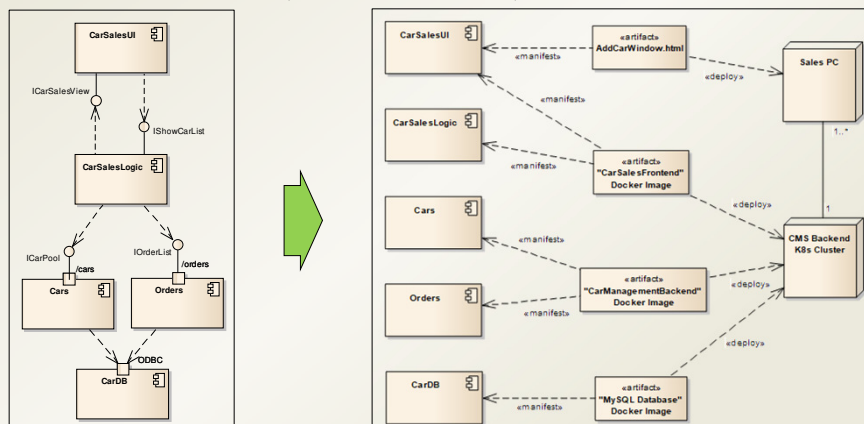
4

Warstwy fizyczne a warstwy logiczne

- Architektura klient-serwer (w uproszczeniu)
 - „Klient” - maszyna (węzeł), która ma bezpośrednią interakcję z użytkownikiem
 - „Serwer” - maszyna, która dostarcza różnych usług i zasobów maszynom klienckim
- „Gruby (ciężki) klient”
 - Klient zapewnia bogatą funkcjonalność bez znaczącego udziału serwera; minimalizacja komunikacji z serwerem
- „Cienki (lekki) klient”
 - Większość funkcjonalności zapewnia serwer; klient służy jedynie do obsługi interakcji z użytkownikiem; bardzo częsta komunikacja klienta z serwerem
- Jak przedstawić grubego i cienkiego klienta w UML’u?

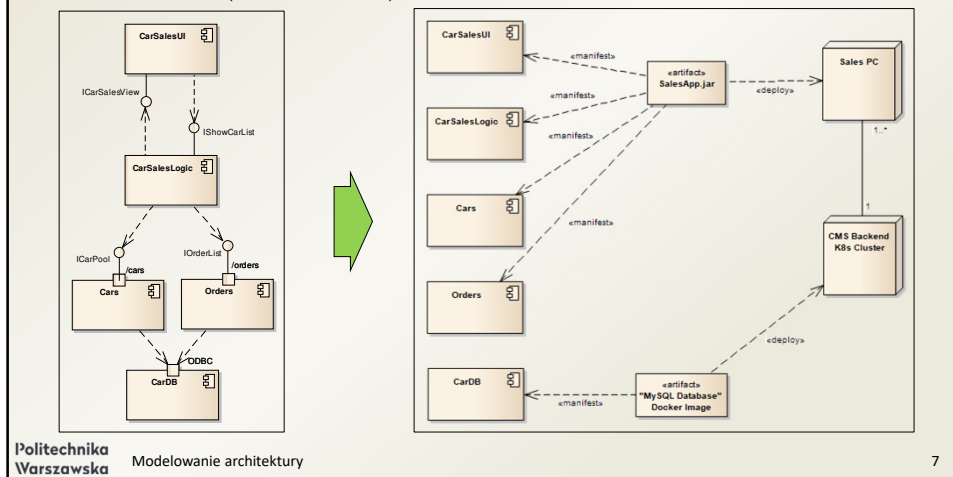
Architektura klient-serwer w języku UML (1)

- Cienki klient (przykład)
 - Warstwy V+P+M+DB w większości „zamontowane” na węźle serwera („CMS Backend”)



Architektura klient-serwer w języku UML (2)

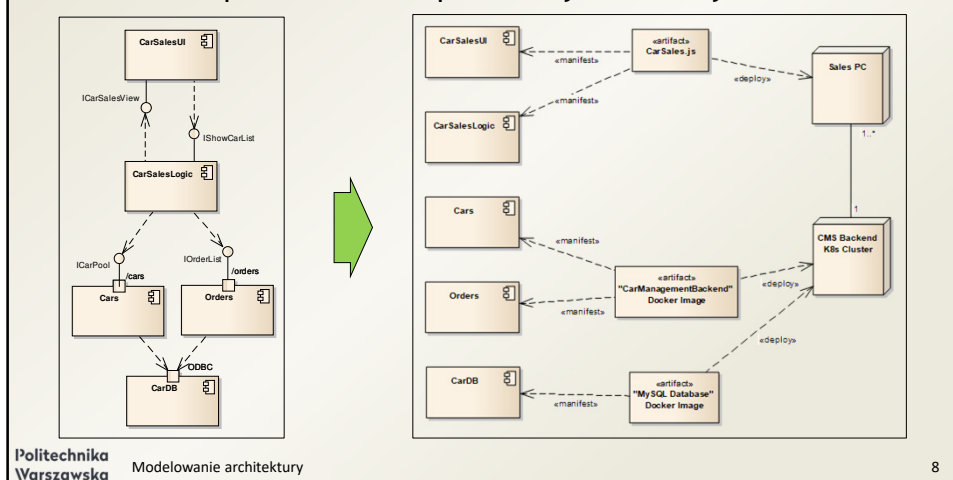
- Gruby klient (przykład)
 - Warstwy V+P+M w większości „zamontowane” na węźle klienta („Sales PC”)



7

Typowa architektura webowa w języku UML

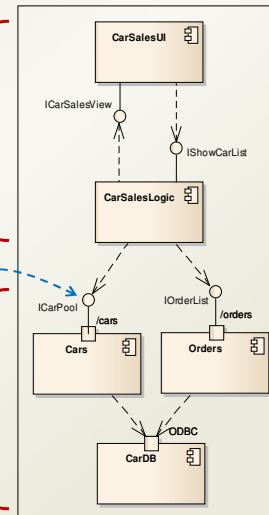
- Podział odpowiedzialności między klienta i serwera
 - Klient: logika aplikacji oraz ew. wstępna walidacja danych
 - Serwer: przetwarzanie i przechowywanie danych



8

Stos technologiczny aplikacji webowych

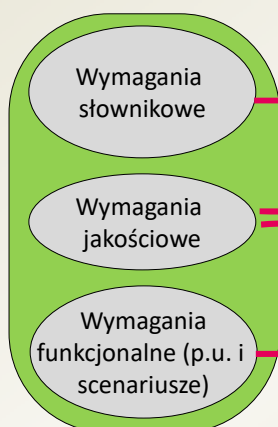
- Komponenty „front-end”: warstwy „V” i „P”
 - Technologie wymiany danych z użytkownikiem i sterowania prezentacją danych
 - Najczęściej: język JavaScript i frameworki Angular, React, Vue, ...
- Komponenty „back-end”: usługi warstwy „M” oraz DB
 - Dostęp do usług: REST, GRPC
 - Technologie przetwarzania, dostępu i przechowywania danych
 - Najczęściej: języki Java, C#, Python oraz frameworki Java EE, Spring, .NET
 - Relacyjne i nierelacyjne bazy danych
 - Technologie ORM (Object-Relational Mapping)



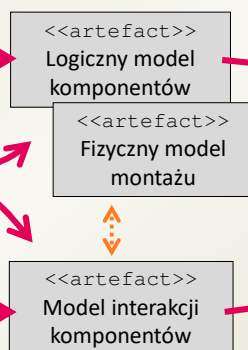
Projektowanie architektury na podstawie wymagań

- Modele architektoniczne wynikają z wymagań i wpływają na projekt kodu

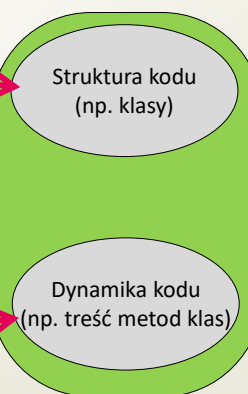
Wymagania



Architektura



Projekt kodu

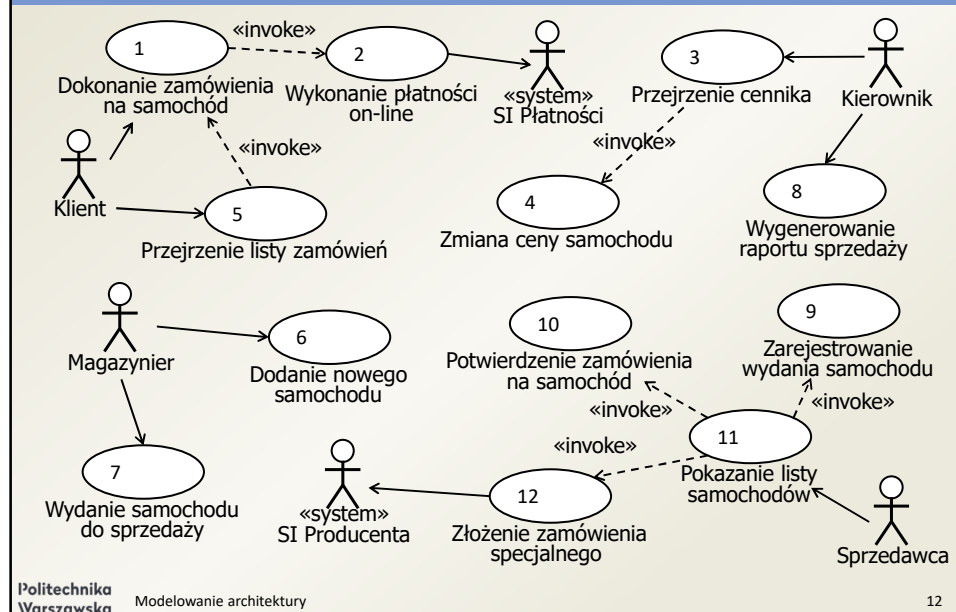


Zasady tworzenia modeli architektonicznych

- Heurystyki (praktyki wyniesione z doświadczeń)
 - zasady „automatyzacji” przejścia od wymagań do projektu architektonicznego
- Decyzje architektoniczne
 - Na bazie wiedzy architekta o technologii i zastosowaniu
 - Wybór najbardziej adekwatnych rozwiązań
- Reguły przejścia od wymagań do architektury
 - Modele źródłowe: pakiety przypadków użycia, przypadki użycia, scenariusze, pakiety klas-pojęć, klasy-pojęcia
 - Modele docelowe: komponenty front-end, komponenty back-end, interfejsy, obiekty transferu danych (DTO)
- **Uwaga:** tu zajmujemy się tylko architekturą logiczną

11

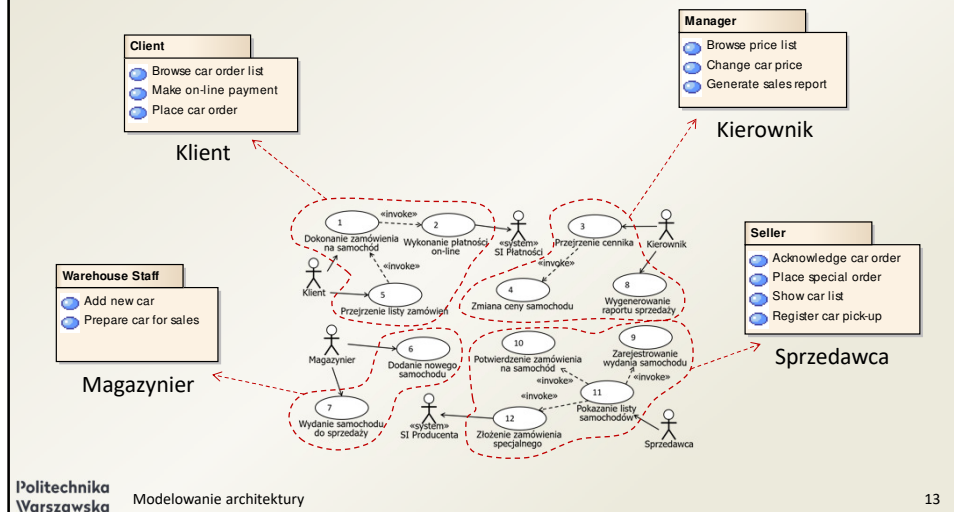
Wymagania: model przypadków użycia (fragment)



12

Pakietyzacja wymagań funkcjonalnych (1)

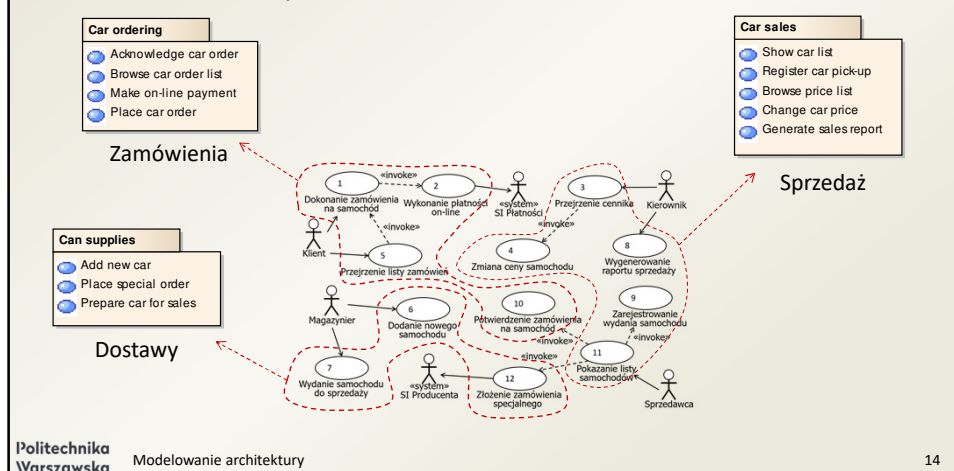
- Pakietyzacja przypadków użycia według głównych aktorów



13

Pakietyzacja wymagań funkcjonalnych (2)

- Pakietyzacja p.u. według obszarów tematycznych – Funkcjonalność powiązana z przetwarzaniem pewnego zakresu danych

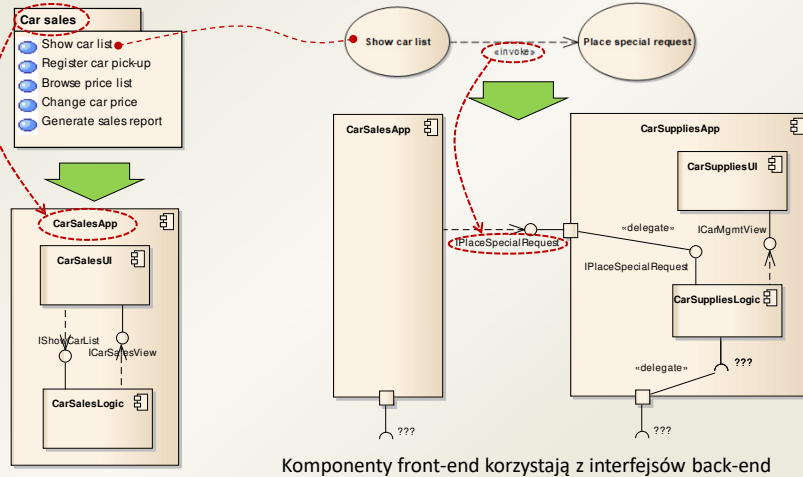


14

Od przypadków użycia do komponentów front-end

Reguła 1: pakiet przypadków użycia →
→ komponent warstwy front-end

Reguła 2: relacja „invoke” (różne pakiety) →
→ interfejs między komponentami f-e



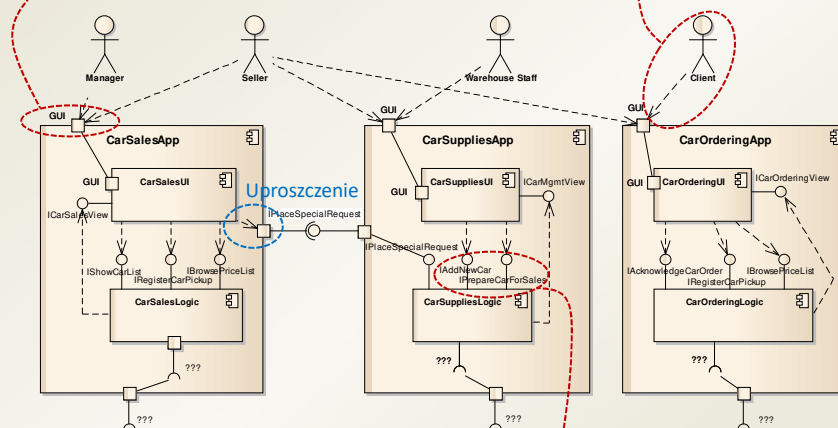
Komponenty front-end korzystają z interfejsów back-end

15

Model komponentów dla warstwy front-end

GUI modelujemy np. jako port

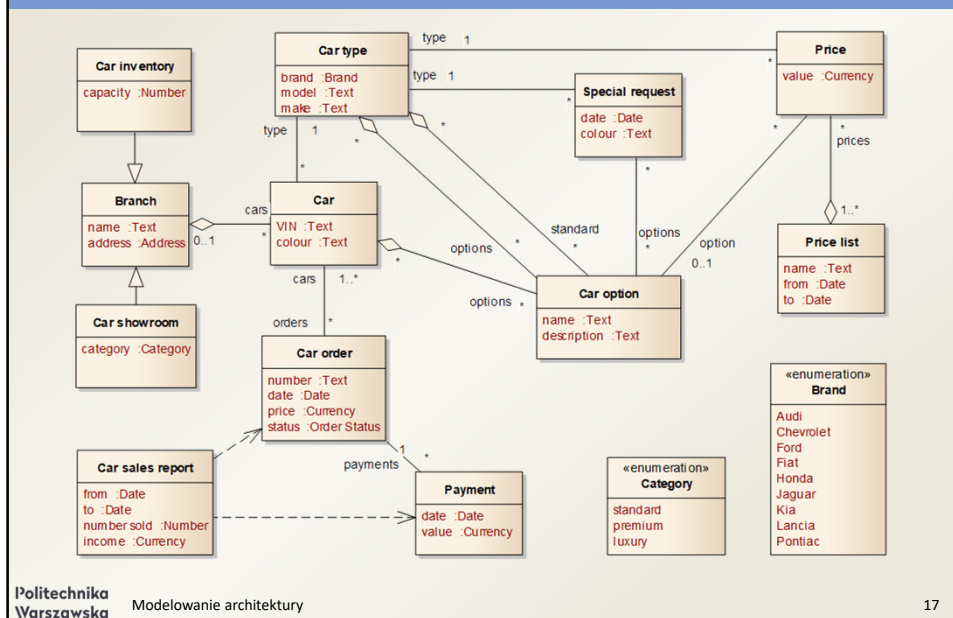
Aktorzy korzystają z komponentów f-e poprzez GUI



Reguła 3: przypadek użycia →
→ wewnętrzny interfejs w komponentach f-e

16

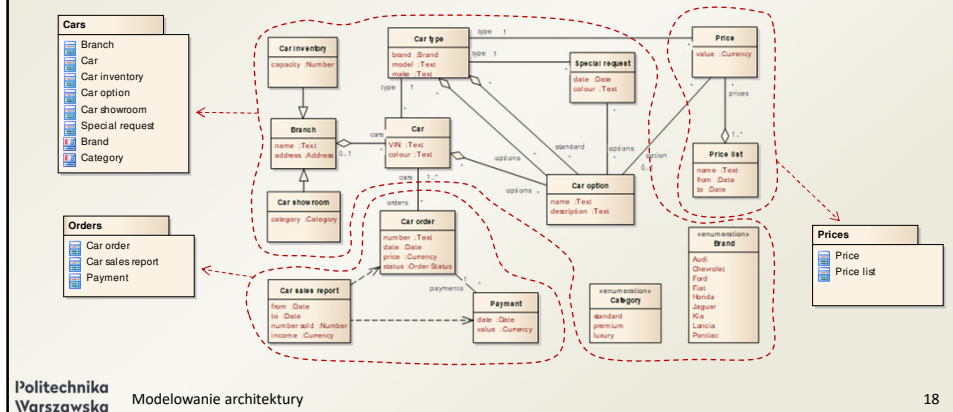
Wymagania: model dziedziny (fragment)



17

Pakietyzacja modelu dziedziny

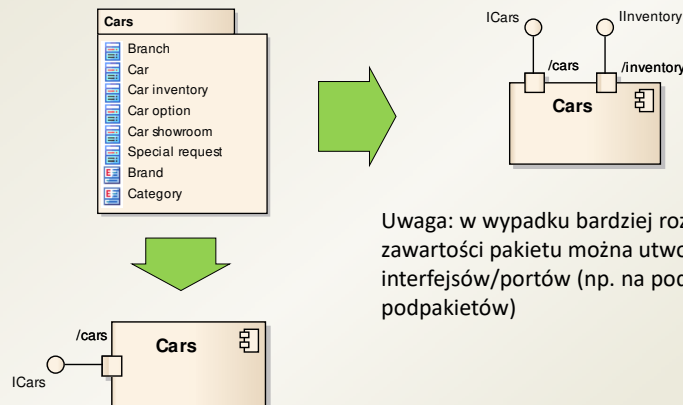
- Pakietyzacja według obszarów dziedziny
 - Kryterium: przechowywane dane („statyka”)
- Pakietyzacja według obszarów funkcjonalnych
 - Kryterium: operacje na danych („dynamika”)



18

Od klas-pojęć do komponentów back-end

Reguła 4: pakiet klas-pojęć →
→ komponent b-e + interfejs(y) dostarczany(e) + port(y)



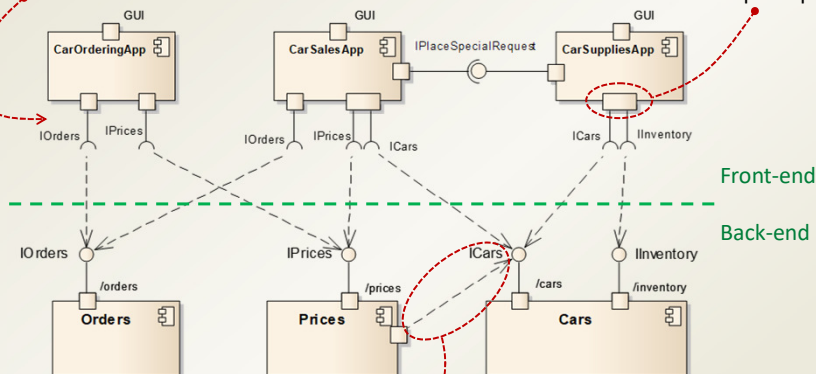
Uwaga: w wypadku bardziej rozbudowanej zawartości pakietu można utworzyć więcej interfejsów/portów (np. na podstawie podpakietów)

19

Model komponentów dla warstwy back-end

Wykorzystanie interfejsów b-e przez f-e zależne od logiki aplikacji

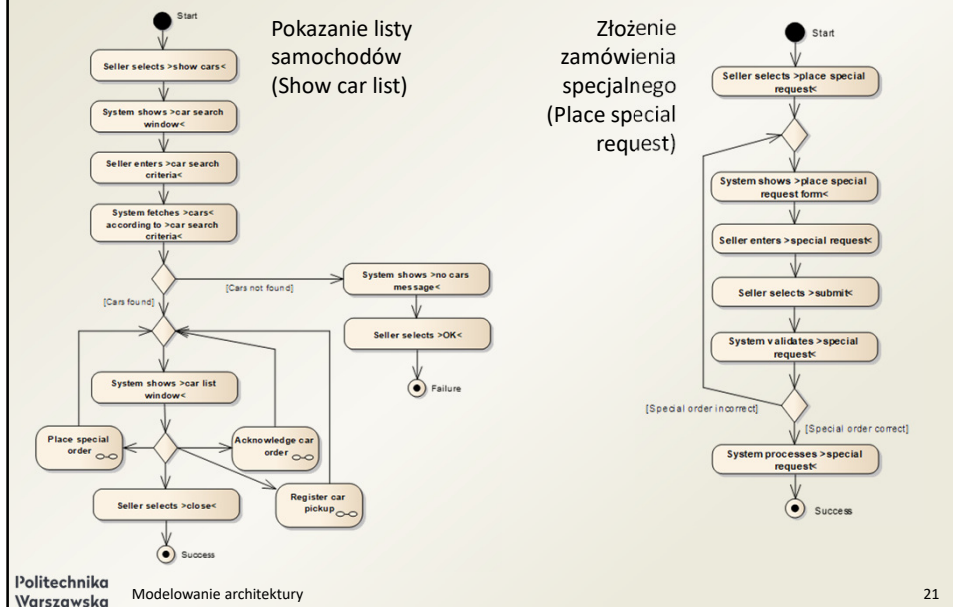
Grupowanie interfejsów przez porty



Reguła 5: silnie powiązanie funkcjonalne klas-pojęć w różnych pakietach
→ wykorzystanie interfejsu pomiędzy komponentami b-e

20

Wymagania: scenariusze przypadków użycia



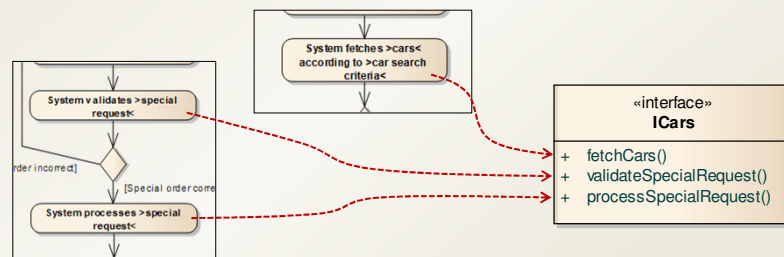
21

Od zdań scenariuszy do operacji interfejsów

Reguła 6: zdania Aktor-do-Systemu → operacja w interfejsie logiki aplikacji



Reguła 7: zdania System-do-Systemu → operacja w interfejsie logiki dziedziny

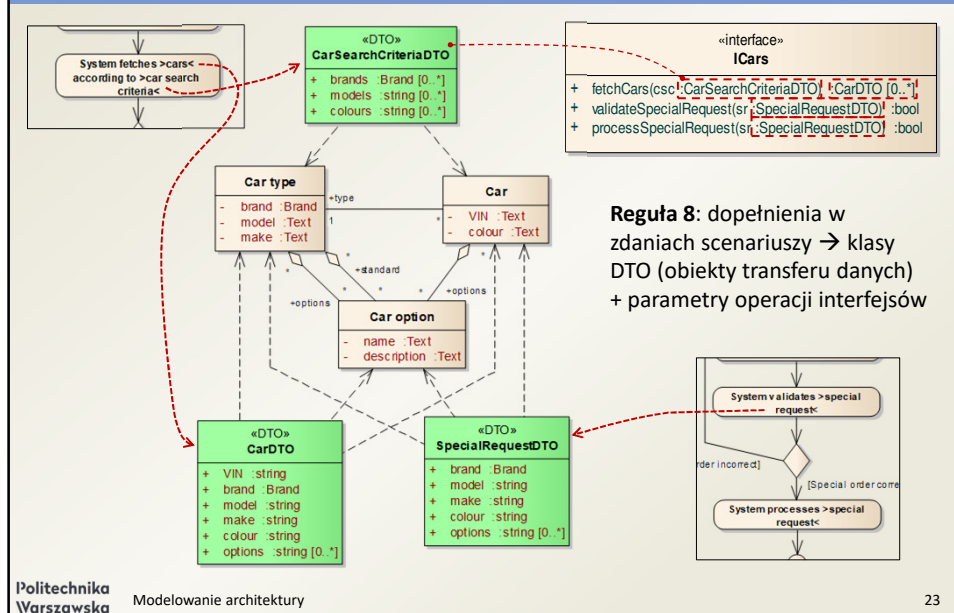


Politechnika Warszawska Modelowanie architektury

22

22

Od zdań scenariuszy do DTO

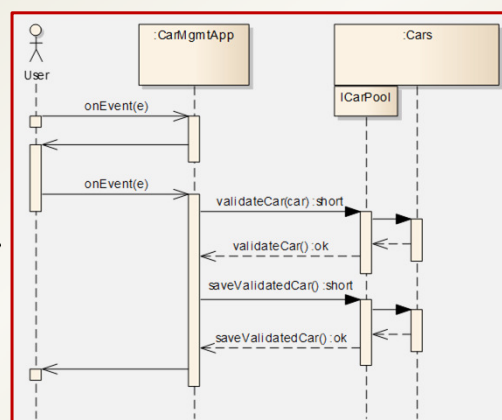


23

Przypomnienie: od scenariusza do diagramu sekwencji

- Jak zobrazować dynamikę dla scenariuszy?

1. Użytkownik wybiera opcję menu „Dodaj samochód”
2. System pokazuje okno dodania samochodu
3. Uż. wprowadza dane samochodu
4. Uż. wybiera „Dodaj”
5. System sprawdza poprawność danych samochodu
6. System zapamiętuje dane samochodu
7. System pokazuje okno potwierdzenia
8. Uż. wybiera „OK”



Uwaga: diagramy sekwencji warto tworzyć dla bardziej złożonych przypadków użycia

24